

APPLICATION PERFORMANCE MONITORING

CERTIFICATION  
**STUDY GUIDE**

Full Edition

# Datadog APM & Distributed Tracing

---

A complete, exam-focused guide to modern application observability, the Datadog APM architecture, trace ingestion & sampling, OpenTelemetry, and service dependency analysis — with diagrams, hands-on labs, solutions, and review questions.

Observability & the Three Pillars

Trace Agent Architecture

Spans & Context Propagation

Head vs Tail Sampling

OpenTelemetry Integration

Service Maps

Labs + Solutions

Prepared for: Vivek Arora

Format: Certification preparation • Full guide

Modules: 3 • Labs: 6 • Review questions: 25

Version 1.0 — August 2026

# Table of Contents

How this guide is organised. Each module ends with hands-on labs (with full solutions), real-world use cases, and a set of certification-style review questions.

|                                                                |    |
|----------------------------------------------------------------|----|
| <b>How to use this guide</b>                                   | 3  |
| <b>Module 1 — Modern Application Observability</b>             | 4  |
| 1.1 The evolution of monitoring                                | 4  |
| 1.2 The three pillars: metrics, logs, traces                   | 5  |
| 1.3 Why APM matters & the business impact of performance       | 7  |
| 1.4 Labs, use cases & review questions                         | 9  |
| <b>Module 2 — Datadog APM Architecture</b>                     | 12 |
| 2.1 APM components & the Trace Agent pipeline                  | 12 |
| 2.2 Trace lifecycle: spans, context propagation                | 14 |
| 2.3 Ingestion & sampling strategies                            | 16 |
| 2.4 OpenTelemetry integration                                  | 19 |
| 2.5 Labs, use cases & review questions                         | 21 |
| <b>Module 3 — Service Dependency Analysis</b>                  | 25 |
| 3.1 The Service Map & service catalog                          | 25 |
| 3.2 Reading dependencies & root-cause workflows                | 26 |
| 3.3 Labs, use cases & review questions                         | 28 |
| <b>Appendix A — Answer keys</b>                                | 30 |
| <b>Appendix B — Glossary &amp; quick-reference cheat sheet</b> | 31 |
| <b>Appendix C — Sources &amp; further reading</b>              | 33 |

# How to use this guide

This guide is written for certification preparation. It builds concepts from first principles, then reinforces them with diagrams, labs, and exam-style questions.

Work through the modules in order — later concepts (sampling, service maps) assume the earlier ones (spans, traces, the agent pipeline). For each module:

- **Read the concept sections** and study the accompanying diagram before moving on. The diagrams are designed to be memorised as mental models for the exam.
- **Watch for the coloured callout boxes.** **Definition** boxes contain terms you must be able to state precisely; **Exam focus** boxes flag facts that are commonly tested; **Tip** boxes give practical guidance; **Watch out** boxes flag common mistakes.
- **Do the labs.** Each lab has a stated goal, step-by-step tasks, and a full worked solution. Attempt the tasks before reading the solution.
- **Answer the review questions** at the end of each module, then check yourself against the answer key in Appendix A.

## A note on screenshots

Where a screenshot of the Datadog UI would appear, this guide uses a labelled figure placeholder (for example, *Figure — Service Map, production environment*) describing exactly what to look at. Because product UIs change frequently, capture a live screenshot from your own Datadog account at that step and paste it in — this keeps your study copy current and matches what you will see in the exam environment.

## Versions & defaults

Product defaults (target traces-per-second, environment variable names, collector distributions) evolve. This guide reflects Datadog's documented behaviour as of mid-2026; always confirm the current default in the official docs (Appendix C) before relying on a specific number in production.

# Modern Application Observability

Where observability came from, the three pillars that make it work, and why application performance is a business concern — not just an engineering one.

## 1.1 The evolution of monitoring

Monitoring has moved through four broad eras. Each era did not replace the previous one — it *added a layer of context* on top of it. Understanding this progression explains why "observability" is treated as distinct from traditional "monitoring."

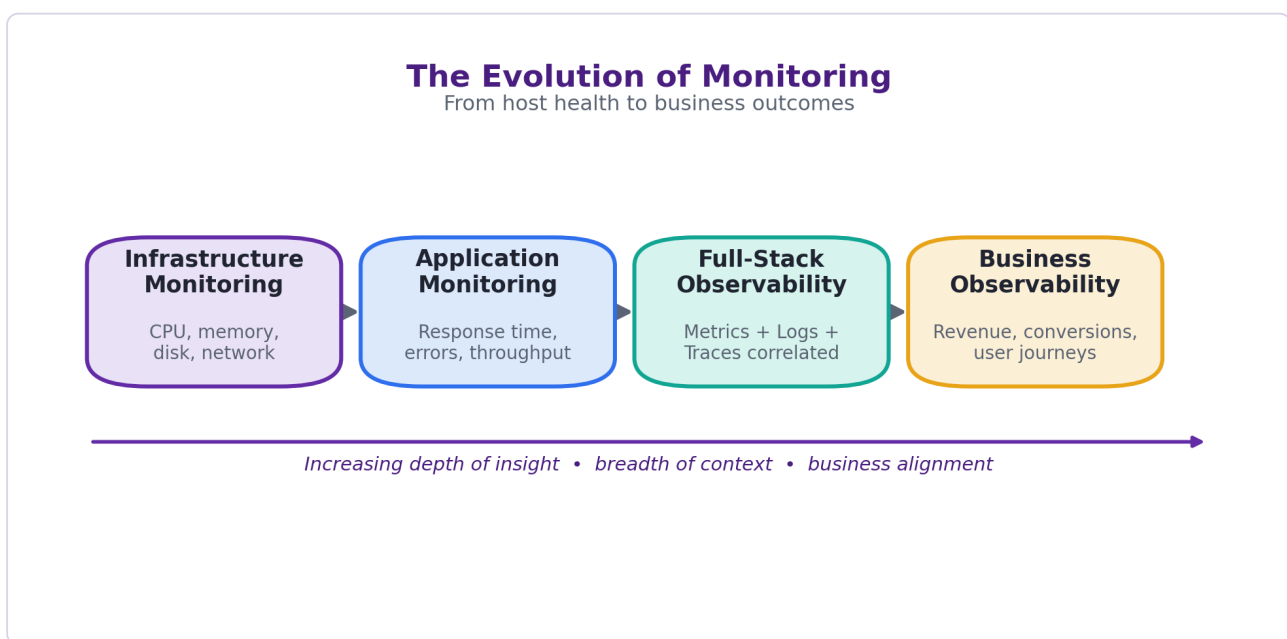


Figure 1.1 — The maturity path from infrastructure monitoring to business observability.

**Infrastructure monitoring** was the first era: watch the health of hosts and networks — CPU, memory, disk, and bandwidth. It answers "is the machine up?" but says nothing about whether the *application* is serving users well. **Application monitoring (APM)** added request-level signals — response time, throughput, and error rates — so teams could see how the software itself behaved. **Full-stack observability** unified the three telemetry types (metrics, logs, and traces) so an engineer can pivot seamlessly from a symptom to its cause across the whole stack. **Business observability** is the newest layer: it ties technical signals directly to outcomes the business cares about — conversions, revenue, cart value, and user journeys.

### Definition — Monitoring vs Observability

**Monitoring** is collecting and alerting on a *predefined* set of signals — you decide in advance what to watch. **Observability** is the property of a system whose internal state you can *infer from its external outputs*, letting you ask new questions about unforeseen problems without shipping new code. Monitoring tells you *that* something is wrong; observability helps you understand *why*, even for failures you did not anticipate.

### Exam focus

Be ready to distinguish "known-unknowns" (monitoring — dashboards and alerts for conditions you predicted) from "unknown-unknowns" (observability — exploring high-cardinality data to explain novel failures). APM is the bridge: it provides request-level, high-cardinality trace data that turns raw monitoring into true observability.

## 1.2 The three pillars: metrics, logs, and traces

Observability is conventionally described through three complementary telemetry types — the "three pillars." No single pillar is sufficient on its own; their power comes from being *correlated*.

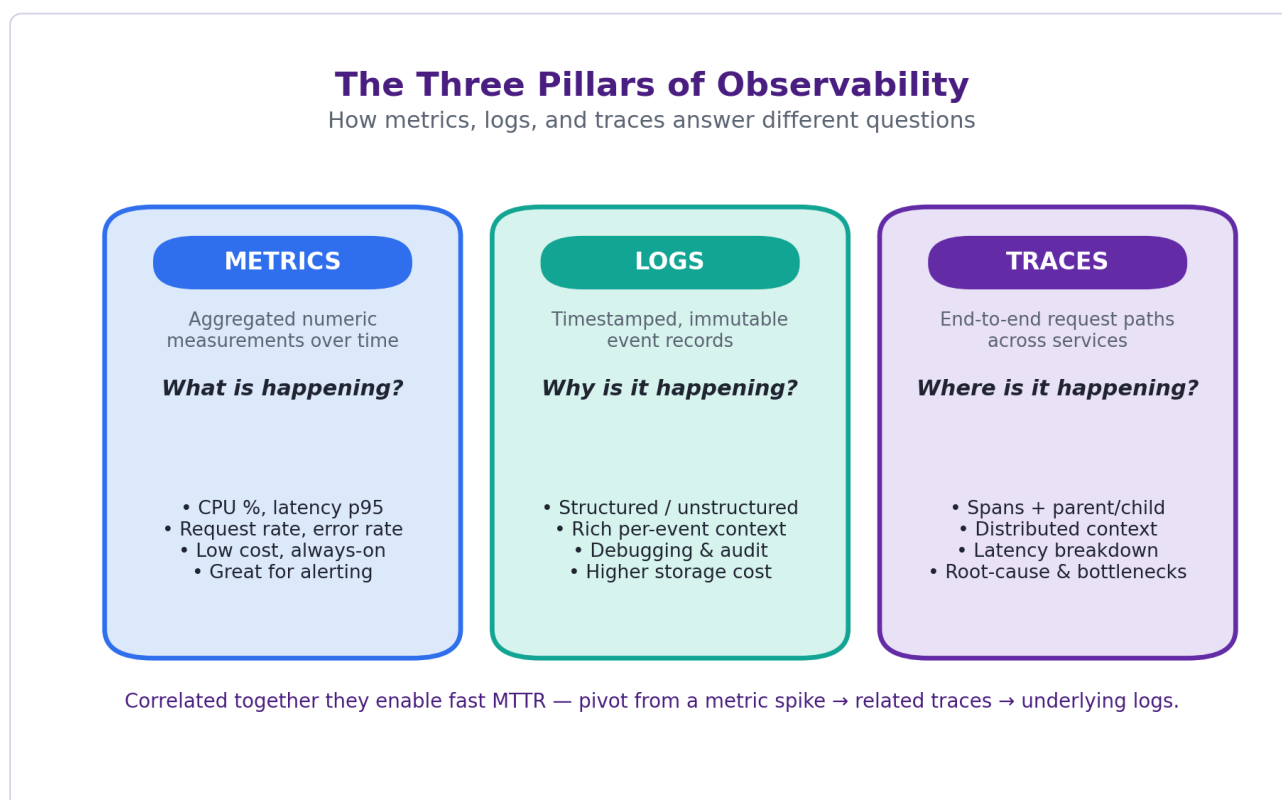


Figure 1.2 — Metrics, logs, and traces answer different questions and carry different cost/detail trade-offs.

**Metrics** are numeric measurements aggregated over time — request rate, error rate, p95 latency, saturation. They are cheap to store, can be retained for long windows, and are ideal for dashboards and alerting because they are compact and mathematically aggregatable. Their weakness is a lack of per-event context: a metric tells you latency rose, not *which* requests were slow or why.

**Logs** are timestamped, immutable records of discrete events. They carry rich, per-event context (a stack trace, a user ID, a SQL statement) and are indispensable for debugging and audit. The trade-off is volume and cost: logging everything at high traffic is expensive, which is why structured logging and log-management pipelines matter.

**Traces** follow a single request end-to-end as it travels across services. A trace is a tree of **spans**, each representing one unit of work (an HTTP handler, a database query, a cache lookup). Traces are the pillar that reveals *where* time is spent in a distributed system and how services depend on one another — they are the foundation of APM and of the service maps covered in Module 3.

| Pillar         | Answers                                      | Best for                                      | Cost / detail                              | Datadog product                     |
|----------------|----------------------------------------------|-----------------------------------------------|--------------------------------------------|-------------------------------------|
| <b>Metrics</b> | What is happening?<br>(trend, magnitude)     | Dashboards, SLOs,<br>alerting                 | Low cost, low<br>detail, high<br>retention | Metrics,<br>Dashboards,<br>Monitors |
| <b>Logs</b>    | Why did this specific<br>event happen?       | Debugging, forensics,<br>audit                | High detail, higher<br>storage cost        | Log Management                      |
| <b>Traces</b>  | Where in the request<br>path is the problem? | Latency analysis,<br>dependencies, root cause | High detail,<br>sampled to control<br>cost | APM & Distributed<br>Tracing        |

#### Tip — Correlation is the real value

In Datadog, the three pillars are linked. A latency spike on a metric dashboard links to the exact *traces* that were slow; each slow trace links to the *logs* emitted during that request (via injected `trace_id`); and each service in a trace links back to its host *infrastructure metrics*. This "trace → logs → metrics" pivot is what collapses mean-time-to-resolution (MTTR).

#### Definition — Cardinality

**Cardinality** is the number of unique values a dimension (tag) can take. `host` is low cardinality; `user_id` or `trace_id` is high cardinality. Metrics struggle with high-cardinality tags (each combination is a separate time series and drives cost), whereas traces are designed for high cardinality — this is precisely why traces answer questions metrics cannot.

## 1.3 Why APM matters — the business impact of performance

Application performance is not an abstract engineering metric; it maps directly onto revenue and retention. Slow or failing requests degrade the user experience, which drives abandonment, which erodes conversions and revenue and increases churn.

## Why APM Matters: Performance Drives Business Outcomes

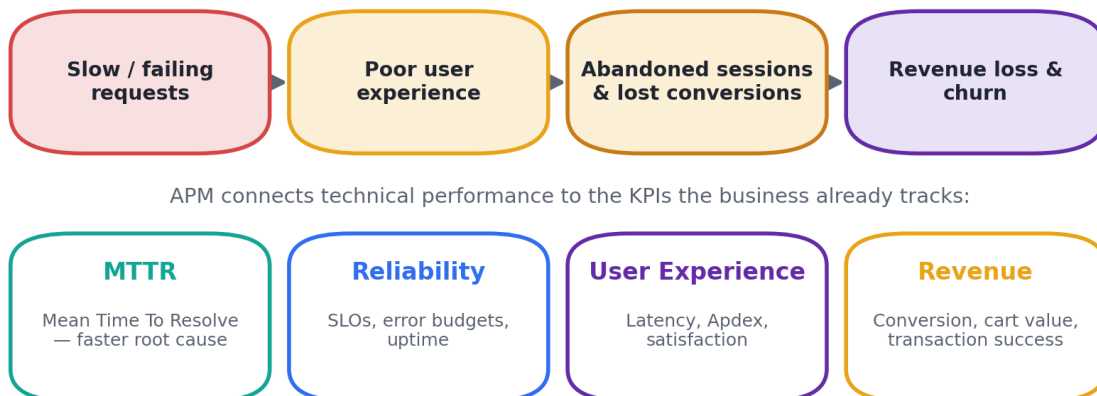


Figure 1.3 — Performance problems propagate into business outcomes; APM connects technical signals to the KPIs the business already tracks.

The widely-cited industry pattern is that added latency depresses engagement and conversion: incremental delays measurably reduce the share of users who complete a purchase or a sign-up, and outright failures (5xx errors, failed payments) directly forfeit transactions. Conversely, faster, more reliable applications retain users and protect revenue. APM makes this link observable and actionable in four areas:

| Business lever                     | What APM provides                                                                     | Representative signal                                              |
|------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>User experience</b>             | Real, per-request latency and error visibility, plus Apdex-style satisfaction scoring | p75 / p95 / p99 latency, Apdex, error rate by endpoint             |
| <b>Revenue &amp; conversion</b>    | Correlation of endpoint performance with funnel steps and failed transactions         | Checkout latency vs conversion; failed <code>charge()</code> spans |
| <b>MTTR (mean time to resolve)</b> | Fast root-cause via traces, flame graphs, and the service map                         | Time from alert → identified culprit service                       |
| <b>Service reliability</b>         | SLOs, error budgets, and dependency awareness                                         | Uptime, error budget burn rate, upstream/downstream health         |

### Definition — MTTR, SLO, SLI & error budget

**MTTR** is the mean time to resolve (or recover from) an incident — a core reliability KPI that good observability shrinks. An **SLI** (Service Level Indicator) is a measured signal such as request-success ratio or latency; an **SLO** (Service Level Objective) is a target for that SLI (e.g. "99.9% of requests succeed"); the **error budget** is the allowed shortfall (the remaining 0.1%) that teams may "spend" on risk before freezing changes.

### Exam focus

Expect a question framing APM's value in business terms. The defensible answer ties the three pillars to *reduced MTTR* and *protected revenue*: traces localise the failing service, logs explain the failure, and metrics/SLOs quantify the user impact and error-budget burn.

## 1.4 Labs, use cases & review questions

### Use case — E-commerce checkout degradation

A retailer sees conversion drop 4% between 18:00 and 20:00 daily. Metrics show overall latency is "normal," so infrastructure monitoring alone finds nothing. With APM, the team filters the `checkout` service to p95 latency and discovers the `payment-service` span is slow only under peak load. Traces reveal a connection-pool exhaustion against the payments provider; logs on those traces show timeout exceptions. The fix (larger pool + circuit breaker) restores conversion. *Lesson*: aggregate metrics hid a tail-latency problem that per-request traces exposed.

#### Lab 1.1 — Map a user problem to the three pillars

*Goal: practise choosing the right telemetry for a given question.*

For each business question below, state which pillar (metrics, logs, or traces) you would reach for *first*, and why:

1. "Is our overall error rate trending up this week?"
2. "Why did *this specific* order #48213 fail at 14:07?"
3. "Which downstream service is adding the most latency to `/checkout`?"
4. "Are we about to breach our 99.9% availability SLO this month?"

#### Solution

(1) **Metrics** — a trend over time across many requests is exactly what aggregated metrics are for; build a monitor on error rate. (2) **Logs** — a single, specific event needs the rich per-event context (stack trace, exception, order ID) that only logs carry; search logs by order ID or by the request's `trace_id`. (3) **Traces** — attributing latency across services is the defining trace question; open the flame graph for slow `/checkout` traces and read the span durations. (4) **Metrics/SLOs** — availability against a target is an SLI tracked as a metric; check the SLO's error-budget burn rate. Note how (2) and (3) are linked: the slow trace from (3) will carry the `trace_id` you use to find the logs in (2).



## Lab 1.2 — Build the correlation pivot mentally

*Goal: internalise the "metric → trace → log" root-cause pivot used throughout Datadog.*

A dashboard shows p99 latency on the `web-store` service jumped from 200 ms to 1.8 s at 09:15. Write, in order, the three pivots you would perform and what you expect to learn at each step.

### Solution

**Pivot 1 (metric → traces):** from the latency graph, jump to APM traces for `web-store` scoped to 09:15 and sorted by duration. Expected: a cluster of slow traces sharing a common slow span. **Pivot 2 (trace → span):** open a representative slow trace's flame graph and find the span consuming the added ~1.6 s — say a `postgres` query span. Expected: the bottleneck service and operation. **Pivot 3 (span → logs):** from that span, open correlated logs (linked by `trace_id`). Expected: log lines showing lock waits or a missing index, i.e. the concrete cause. Total path: symptom → location → explanation, with each step narrowing scope.

## Review questions — Module 1

- Q1.** In one sentence, what is the key difference between *monitoring* and *observability*?
- Q2.** Name the three pillars of observability and give the primary question each one answers.
- Q3.** Why are traces better suited to high-cardinality data (like `user_id`) than metrics are?
- Q4.** A team can only afford to store a fraction of one telemetry type in full. Which pillar is *most* commonly sampled, and why does that not lose trend accuracy? (Hint: think about what statistics the agent computes on 100% of spans.)
- Q5.** Define SLI, SLO, and error budget, and explain how they relate.
- Q6.** Give two concrete ways APM reduces MTTR compared with infrastructure monitoring alone.

Answers in Appendix A.

# Datadog APM Architecture

How a request becomes a trace, how the Datadog Agent processes it, how sampling controls cost, and how OpenTelemetry fits in.

## 2.1 APM components & the Trace Agent pipeline

Datadog APM has three moving parts: **instrumentation** inside your application, the **Trace Agent** (a component of the Datadog Agent) that runs alongside it, and the **Datadog backend** that stores, indexes, and visualises the data.

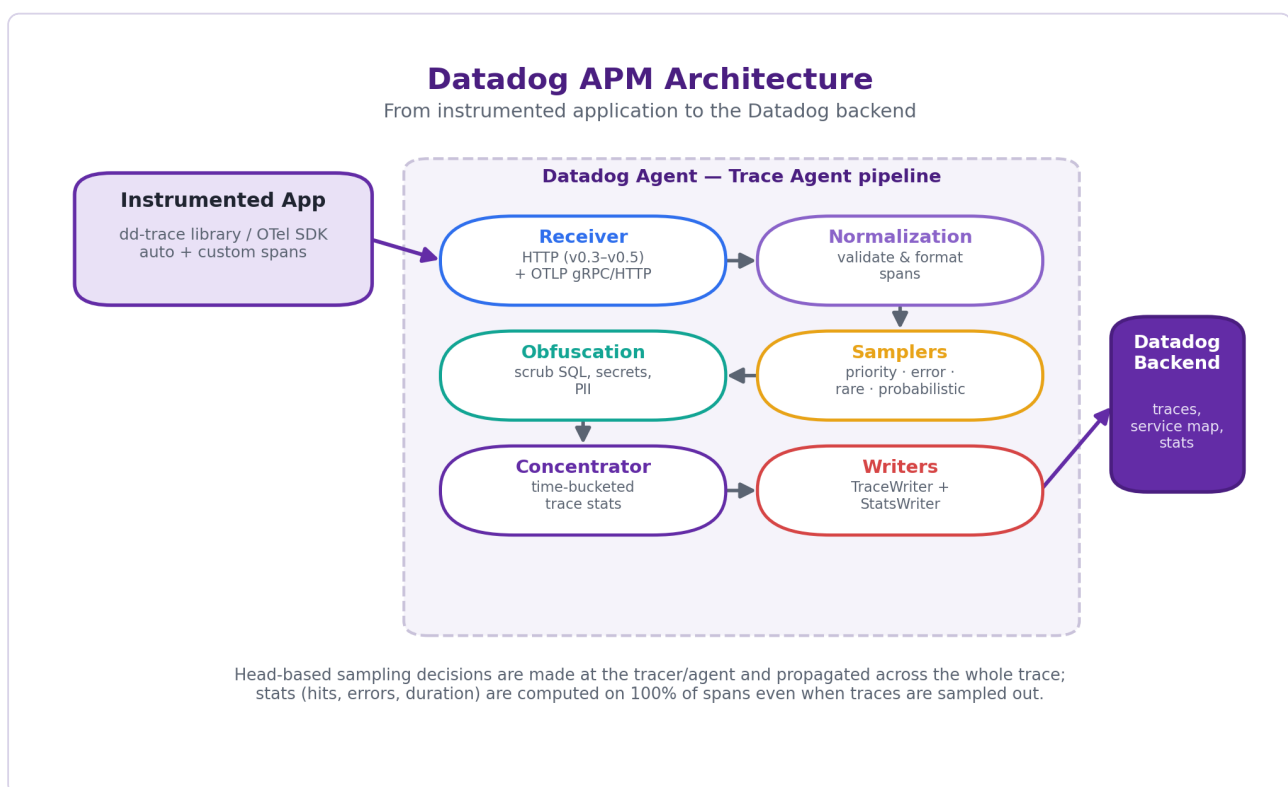


Figure 2.1 — End-to-end path: instrumented app → Trace Agent pipeline (receiver, normalization, obfuscation, samplers, concentrator, writers) → Datadog backend.

**Application instrumentation** is what produces spans. A Datadog tracing library (`dd-trace`, available for Java, Python, Go, Ruby, Node.js, .NET, PHP, C++, and Rust) hooks into common frameworks and clients to *auto-instrument* inbound requests, database calls, HTTP clients, and message queues. You can add *custom spans* for business-specific operations. Alternatively, an OpenTelemetry SDK can produce the spans (see §2.4).

The **Trace Agent** receives spans locally (avoiding sending raw data over the internet from the app itself) and runs them through a pipeline before forwarding to Datadog:

| Stage        | Component                                                      | What it does                                                                                                                                                |
|--------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Receive   | <code>HTTPReceiver</code> / <code>OTLPReceiver</code>          | Accepts spans over the Datadog trace protocol (v0.3–v0.5) or via OTLP (gRPC/HTTP) from OpenTelemetry SDKs.                                                  |
| 2. Normalize | Normalization                                                  | Validates and standardises spans (required fields, types, service/resource names) so downstream stages get consistent data.                                 |
| 3. Obfuscate | Obfuscation engine                                             | Scrubs sensitive content — SQL query literals, credit-card-like numbers, secrets — before anything leaves the host.                                         |
| 4. Sample    | Samplers (priority, error, rare, probabilistic)                | Decide which traces to keep for full storage. See §2.3.                                                                                                     |
| 5. Aggregate | <code>Concentrator</code> / <code>ClientStatsAggregator</code> | Computes time-bucketed <i>trace stats</i> (hit count, error count, duration distributions) — on <b>100%</b> of spans, even those later dropped by sampling. |
| 6. Write     | <code>TraceWriter</code> + <code>StatsWriter</code>            | Forwards sampled traces and aggregated stats to the Datadog intake.                                                                                         |

### Exam focus — the "stats vs sampling" distinction

A frequently-tested subtlety: **trace metrics/stats are computed on every span** before sampling, while **full traces are sampled** to control storage cost. This is why your request-count and error-rate metrics stay accurate even when you retain only, say, 10% of full traces. Losing a trace to sampling does *not* mean losing that request from your APM metrics.

### Definition — Agent-side vs library-side

Some decisions happen in the **tracing library** inside your app (creating spans, applying `DD_TRACE_SAMPLING_RULES`, propagating context); others happen in the **Trace Agent** (obfuscation, the auto/target-TPS sampler, error & rare samplers, stats aggregation). Knowing which side owns a behaviour explains where you configure it.

## Deployment patterns

The Agent runs where your workloads run: as a host agent (one per VM), as a **DaemonSet** in Kubernetes (one agent per node, apps send to the node's agent), or as a sidecar. Applications send traces to the local agent over `localhost` (default trace port `8126`) or a Unix domain socket.

```
# Minimal app-side configuration (environment variables)
DD_ENV=prod           # unified service tagging: env
DD_SERVICE=web-store  # service name shown in APM
DD_VERSION=1.4.2      # enables deployment tracking
DD_AGENT_HOST=localhost
DD_TRACE_AGENT_PORT=8126
```

### Definition — Unified Service Tagging

Datadog correlates traces, metrics, and logs using three reserved tags: `env`, `service`, and `version`. Setting these consistently (via `DD_ENV`, `DD_SERVICE`, `DD_VERSION`) is what makes cross-pillar correlation and deployment tracking work. `version` in particular powers "deployment" comparisons — spotting a regression introduced by a specific release.

## 2.2 Trace lifecycle: spans, context propagation & relationships

A **trace** is the full journey of one request; a **span** is one timed operation within it. Spans form a tree via parent/child links, and the whole tree shares a single `trace_id`.

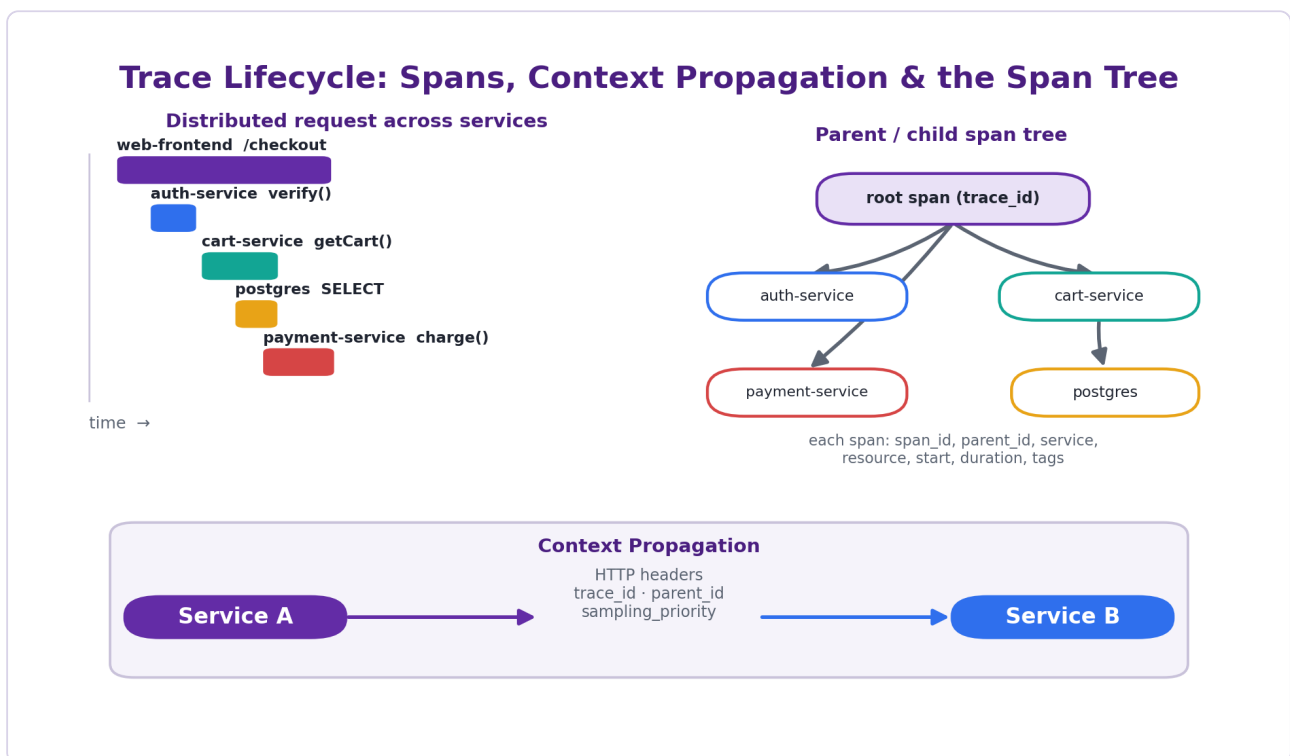


Figure 2.2 — Left: a request's spans on a timeline (flame graph). Right: the same spans as a parent/child tree. Bottom: context propagation carries trace identifiers across service boundaries.

### Definition — Span, trace, root span

A **span** records: `span_id`, `parent_id` (empty for the root), `trace_id`, `service`, `resource` (the specific operation, e.g. `GET /checkout` or a SQL query), operation name, start time, duration, and key-value *tags* and *metrics*. A **trace** is all spans sharing a `trace_id`. The **root span** is the first span (no parent) and is where head-based sampling decisions originate.

**Context propagation** is how the trace stays connected as the request crosses process and network boundaries. When service A calls service B, the tracing library injects trace context into the outbound request — traditionally via HTTP headers — so B can create *child* spans under the same trace. The propagated context carries at minimum the `trace_id`, the caller's `span_id` (becoming the child's `parent_id`), and the **sampling priority** (the keep/drop decision).

| Propagation style | Header(s)                                                                                                     | Notes                                                                            |
|-------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Datadog           | <code>x-datadog-trace-id</code> , <code>x-datadog-parent-id</code> , <code>x-datadog-sampling-priority</code> | Native Datadog format.                                                           |
| W3C Trace Context | <code>traceparent</code> , <code>tracestate</code>                                                            | Vendor-neutral standard; the default/interop format and what OpenTelemetry uses. |
| B3 (Zipkin)       | <code>b3</code> / <code>x-b3-*</code>                                                                         | Common in older/OSS meshes.                                                      |

#### Watch out — broken traces from header mismatch

If two services use *different* propagation formats and neither is configured to read the other, the downstream service starts a brand-new trace and the request appears as two disconnected traces. Standardising on W3C `tracecontext` (or explicitly listing multiple accepted styles via `DD_TRACE_PROPAGATION_STYLE`) prevents this — a classic real-world and exam gotcha.

#### Definition — Sampling priority

A numeric flag propagated with the trace context telling the whole trace whether it will be kept. Common values: `-1 / 0` = drop (user-reject / auto-reject), `1 / 2` = keep (auto-keep / user-keep). Because it is propagated at the root, every service in the trace honours the same decision — guaranteeing traces are kept or dropped as *complete units*.

## 2.3 Ingestion & sampling strategies

At scale, storing every trace is neither affordable nor necessary. **Ingestion sampling** keeps a representative and *interesting* subset (errors, slow requests, rare paths) while trace *metrics* remain computed on everything. Datadog's default model is **head-based**; more advanced setups can use **tail-based** sampling (typically via the OpenTelemetry Collector).

## Sampling Strategies: Head-Based vs Tail-Based

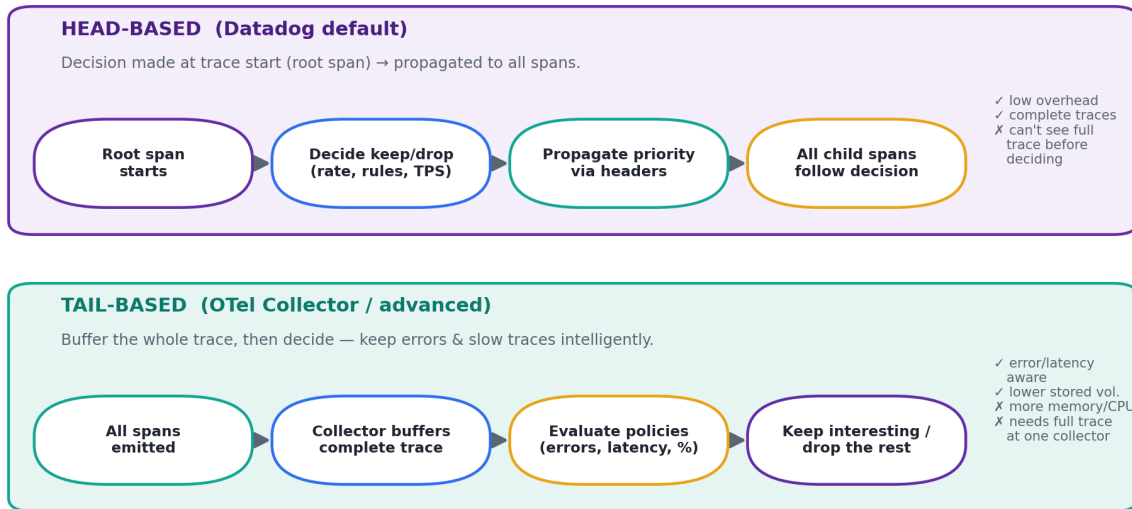


Figure 2.3 — Head-based decides at the start of the trace and propagates the decision; tail-based buffers the whole trace, then decides using error/latency-aware policies.

### Head-based sampling (Datadog default)

The decision is made at the **root span** and propagated to all children via the sampling priority. It is cheap and guarantees complete traces, but it must decide *before* the trace has finished — so it cannot key the decision on, say, the trace's total latency. Datadog layers several mechanisms, each tagged with an `ingestion_reason`:

| Mechanism               | <code>ingestion_reason</code> | Behaviour & key control                                                                                                                                                                |
|-------------------------|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agent auto (target TPS) | <code>auto</code>             | The agent continuously adjusts per-service rates to hit an overall target (default 10 traces/sec). Configure with <code>DD_APM_TARGET_TPS</code> ; adjustable remotely on Agent 7.42+. |
| Library sampling rules  | <code>rule</code>             | Per-service/per-resource rates and rate limits set in the tracer via <code>DD_TRACE_SAMPLING_RULES</code> (default rate limit ~100 traces/sec/instance).                               |
| Error sampler           | <code>error</code>            | Catches error traces missed by head sampling — up to ~10 traces/sec/agent. Control with <code>DD_APM_ERROR_TPS</code> .                                                                |
| Rare sampler            | <code>rare</code>             | Keeps diverse/rare span combinations — up to ~5 traces/sec/agent. Enable with <code>DD_APM_ENABLE_RARE_SAMPLER</code> .                                                                |
| Manual keep / drop      | <code>manual</code>           | Force-keep a critical trace (or force-drop) via a span tag / <code>manual.keep</code> in the tracer.                                                                                   |
| Single span sampling    | <code>single_span</code>      | Keep individual spans from otherwise-dropped traces (e.g. to retain a metric) via <code>DD_SPAN_SAMPLING_RULES</code> .                                                                |

```
# Library sampling rules: keep 100% of checkout, 10% of everything else
DD_TRACE_SAMPLING_RULES='[{"service":"checkout","sample_rate":1.0},
                           {"sample_rate":0.1}]'
# Cap total kept traces per instance
DD_TRACE_RATE_LIMIT=200
```

## Tail-based sampling (advanced)

Tail-based sampling *buffers the complete trace* before deciding, so policies can be error- and latency-aware ("keep all traces with an error, all traces slower than 2 s, and 5% of the rest"). It is typically implemented in the **OpenTelemetry Collector's** `tail_sampling` processor. The trade-off is resource cost (memory to buffer traces) and a hard constraint: *all spans of a trace must reach the same collector instance*, which shapes how you deploy collectors (load-balancing by `trace_id`).

|                                | Head-based                       | Tail-based                                |
|--------------------------------|----------------------------------|-------------------------------------------|
| Decision point                 | Trace start (root span)          | After full trace is buffered              |
| Can use total latency / error? | No (error sampler compensates)   | Yes                                       |
| Overhead                       | Very low                         | Higher CPU/memory                         |
| Deployment constraint          | None                             | Whole trace to one collector              |
| Where                          | Tracer + Datadog Agent (default) | OTel Collector <code>tail_sampling</code> |

### Exam focus

Remember the one-line discriminator: **head-based = decide first, propagate; tail-based = buffer, then decide**. Head-based cannot see the full trace before deciding (so Datadog adds dedicated *error* and *rare* samplers to avoid missing interesting-but-rare traces); tail-based can, at the cost of buffering resources and a same-collector requirement.

### Definition — Ingestion vs indexing (retention filters)

Two separate cost controls. **Ingestion** is how much trace data the agent sends to Datadog (governed by the samplers above). **Indexing** (retention filters) is which *ingested* spans Datadog keeps searchable for 15 days. You can ingest broadly for trace metrics yet index selectively — a common cost-optimisation pattern.

## 2.4 OpenTelemetry (OTel) integration

**OpenTelemetry** is the vendor-neutral CNCF standard for generating and collecting telemetry (an API, SDKs, the OTLP wire protocol, and the Collector). Datadog supports OTel first-class, so teams can instrument once with OTel and still use Datadog's backend and features.

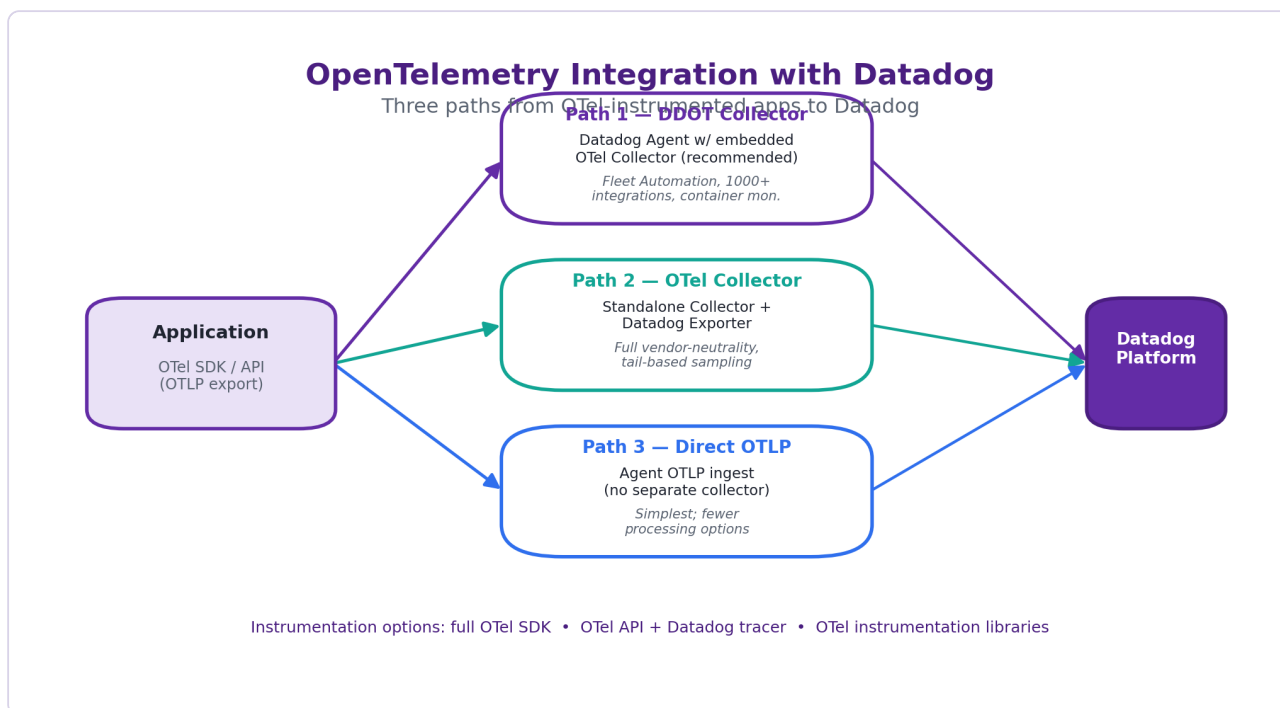


Figure 2.4 — Three supported paths from OTEL-instrumented apps to Datadog.

There are two decisions: *how you instrument* and *how you route* the data.

## Instrumentation choices

- **Full OpenTelemetry** — the OTEL SDK/API produce spans. Maximum vendor-neutrality; some Datadog-specific features may require mapping.
- **OTel API + Datadog tracer** — write instrumentation against the OTEL API but run the Datadog library underneath, unlocking more Datadog features while keeping portable instrumentation.
- **OTel instrumentation libraries** — reuse the OTEL ecosystem's framework instrumentations for broad coverage.

## Routing choices

| Path                                                | What it is                                                                                                                                          | Best when                                                                          |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>DDOT Collector</b><br>(recommended)              | The Datadog Agent with an embedded OpenTelemetry Collector — OTEL flexibility plus Fleet Automation, container monitoring, and 1,000+ integrations. | You want OTEL <i>and</i> the full Datadog ecosystem.                               |
| <b>Standalone OTel Collector + Datadog Exporter</b> | A vendor-neutral Collector that exports to Datadog via the <code>datadog</code> exporter; supports processors like tail-based sampling.             | You need full vendor-neutrality or Collector-side processing (e.g. tail sampling). |
| <b>Direct OTLP ingestion</b>                        | Apps send OTLP straight to the Agent's OTLP endpoint — no separate collector.                                                                       | Simplest setup; fewer in-pipeline processing options.                              |



### Definition — OTel Collector pipeline

The Collector processes telemetry through **receivers** (ingest, e.g. `otlp`) → **processors** (transform/sample, e.g. `batch`, `tail_sampling`) → **exporters** (send onward, e.g. `datadog`). This receiver–processor–exporter model is the mental picture to hold for any Collector question.

```
# OTel Collector: minimal Datadog exporter pipeline
receivers:
  otlp:
    protocols: { grpc: {}, http: {} }
processors:
  batch: {}
exporters:
  datadog:
    api: { key: ${DD_API_KEY}, site: datadoghq.com }
service:
  pipelines:
    traces: { receivers: [otlp], processors: [batch], exporters: [datadog] }
```

### Tip — Native vs OTel: choosing

If you are all-in on Datadog and want the least friction and the richest feature set, native `dd-trace` instrumentation with the Datadog Agent is simplest. If portability across vendors matters, or you need Collector-side processing such as tail-based sampling, choose OpenTelemetry — ideally via the DDOT Collector so you keep Datadog's ecosystem features.

## 2.5 Labs, use cases & review questions

### Use case — Taming trace cost without losing visibility

A platform team ingests 8 billion spans/day and the APM bill is climbing. Rather than blindly cut sampling, they: (1) keep **100% of error and slow traces** using the error sampler plus a tail-sampling policy; (2) drop the bulk of healthy, fast `health-check` traffic; (3) rely on **trace metrics** (computed on 100% of spans) for dashboards and alerts; and (4) tighten **retention filters** so only searched-for spans are indexed. Result: ~70% lower indexed volume with *no* loss of error visibility or metric accuracy — the textbook demonstration of "ingest broadly, index selectively."

## Lab 2.1 — Trace the pipeline for one request

*Goal: cement the Trace Agent pipeline order and what each stage guarantees.*

A `web-store` request calls `auth`, then `cart`, then `payments` (which queries `postgres`). The SQL contains a customer email. The head sampler ultimately drops this trace. Answer:

1. In what order do the pipeline stages process the spans?
2. At which stage is the email removed, and why does it matter that this happens locally?
3. The trace is dropped by sampling — will the request still appear in the `web-store` request-count and error-rate metrics? Explain.

### Solution

(1) Receive → Normalize → Obfuscate → Sample → Aggregate (stats) → Write. (2) The email is scrubbed by the **obfuscation engine** (stage 3), which runs *on the host* before any data leaves — so sensitive literals never traverse the network or reach Datadog's backend. (3) **Yes**. The concentrator computes trace *stats* (hits, errors, durations) on 100% of spans *before* the sampling decision affects storage, so request-count and error-rate remain accurate even though the full trace itself is not retained for search.

## Lab 2.2 — Design a sampling strategy

*Goal: choose and justify sampling mechanisms for real requirements.*

Requirements: (a) keep every trace that contains an error; (b) keep every trace slower than 2 s; (c) always keep 100% of the `checkout` service; (d) keep only ~5% of high-volume `/healthz` traffic; (e) keep overall cost predictable. Specify which mechanism satisfies each, and note where head-based sampling alone is insufficient.

### Solution

(a) The **error sampler** (`DD_APM_ERROR_TPS`) catches error traces head sampling missed; for a true 100% guarantee at high error volume, a **tail-sampling** error policy in the OTel Collector is more complete. (b) Requirement (b) is the key case where **head-based sampling alone cannot help** — you don't know a trace's total latency at the root. Use a **tail-sampling latency policy** (buffer, then keep if duration > 2 s). (c) A **library sampling rule** `{"service":"checkout","sample_rate":1.0}`. (d) A library rule `{"resource":"/healthz","sample_rate":0.05}` (or drop at the agent). (e) Combine per-service rules with an overall **rate limit** / target TPS so total kept traces stay bounded, and use **retention filters** to cap indexed volume. Takeaway: (b) forces tail-based; the rest are satisfiable head-side.

## Lab 2.3 — Fix a broken distributed trace

*Goal: diagnose context-propagation failures.*

After adding a new Go service between `web-store` and `cart`, traces now show `web-store` ending abruptly and a *separate* trace beginning at `cart`. The new service is instrumented and emitting spans. What is the most likely cause, and how do you fix it?

### Solution

The new service is almost certainly **not propagating (or not reading) the same trace-context headers** as its neighbours — e.g. neighbours send W3C `traceparent` but the new service only reads Datadog headers (or vice-versa), so it treats the incoming request as a fresh root and starts a new `trace_id`. Fix by aligning propagation styles across all services — set `DD_TRACE_PROPAGATION_STYLE` (extract and inject) to include the common format (standardise on `tracecontext`), redeploy, and confirm the child spans reattach under one `trace_id`.

## Review questions — Module 2

- Q7. List the six stages of the Trace Agent pipeline in order.
- Q8. Why do APM request/error metrics remain accurate even when most traces are sampled out?
- Q9. What five fields minimally identify a span and its place in the tree?
- Q10. Explain, in one sentence each, head-based vs tail-based sampling.
- Q11. Which single requirement *cannot* be satisfied by head-based sampling, and why?
- Q12. Name the three reserved Unified Service Tagging tags and one thing each enables.
- Q13. What is the deployment constraint that tail-based sampling imposes on OTel Collectors?
- Q14. Describe the OTel Collector's receiver–processor–exporter model and where the `datadog` exporter sits.
- Q15. Give one reason to choose the DDOT Collector over a standalone OTel Collector.
- Q16. A trace splits into two disconnected traces at a service boundary. What is the most likely cause?

Answers in Appendix A.

# Service Dependency Analysis

Turning trace data into a live map of your architecture — and using it to find bottlenecks and root causes fast.

## 3.1 The Service Map & service catalog

Every trace records which service called which. Datadog aggregates these observed calls into a **Service Map**: a live, auto-generated graph of your architecture built entirely from real traffic — no manual diagram to maintain.

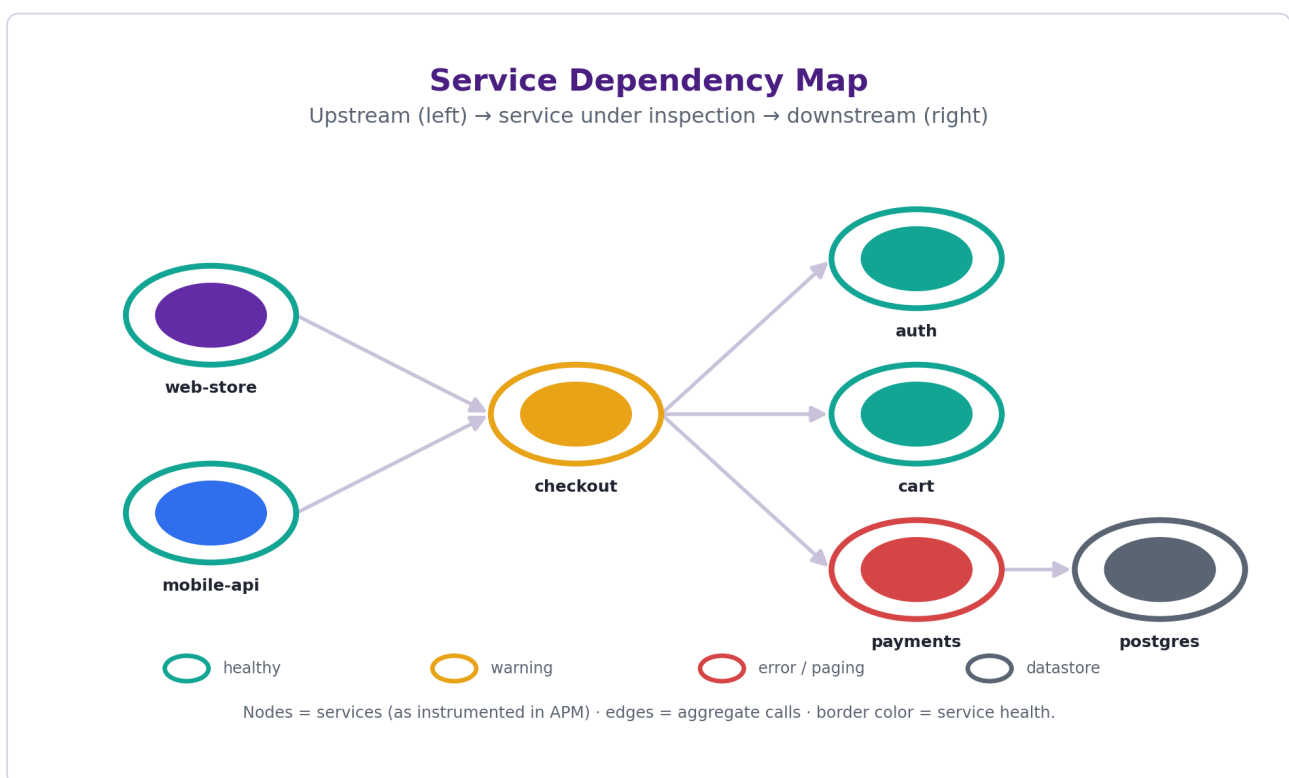


Figure 3.1 — A Service Map: nodes are services, edges are aggregate calls, and node border colour reflects service health. Upstream callers sit left; downstream dependencies sit right.

### Definition — Service Map

A visual graph where **nodes** are services (exactly as instrumented in APM) and **edges** are aggregate calls from one service to another, drawn from observed trace data in real time. Services appear within moments of being instrumented and automatically drop off after ~30 days without traffic. Node borders turn **yellow** or **red** based on *Service Health*, which consolidates Watchdog anomalies, paging monitors, and incidents into a single per-service state.

When you select a single service, the map isolates it to show **upstream** services (who calls it — request sources, drawn to the left) and **downstream** services (what it calls — its dependencies, drawn to the right).

This left-to-right layout mirrors the direction of causality: customer-facing entry points on the left, potential root causes on the right.

The related **Service Catalog** complements the map with an inventory view: ownership (team), on-call, deployment version, dependencies, SLOs, and links to each service's dashboards, monitors, and docs. Where the map is spatial, the catalog is tabular — together they answer "how are things connected?" and "who owns this and is it healthy?"

| Element            | Meaning                                                                       |
|--------------------|-------------------------------------------------------------------------------|
| Node               | A service as instrumented in APM (also datastores/queues if traced).          |
| Edge               | Aggregate calls between two services; thickness/animation can reflect volume. |
| Green / no ring    | Healthy.                                                                      |
| Yellow border      | Warning — anomaly or warn-level monitor.                                      |
| Red border         | Error / active paging monitor or incident.                                    |
| Upstream (left)    | Services that call the selected service.                                      |
| Downstream (right) | Services/datastores the selected service depends on.                          |

## 3.2 Reading dependencies & root-cause workflows

The Service Map is most valuable during incidents, when you must localise a failure across many services quickly. A disciplined reading of the map follows the dependency direction.

### Tip — The "follow the red, go downstream" heuristic

Start at the red customer-facing service (the symptom). Trace edges *downstream* (to the right). A service that is red *and whose own downstream dependencies are all green* is likely the true culprit; a service that is red only because something it depends on is red is a *victim*. You are looking for the deepest red node with healthy dependencies — the failure's origin.

From any node you can pivot directly to that service's traces, its p50/p95/p99 latency and error-rate graphs, its logs, its underlying infrastructure, and its dashboards — the same cross-pillar correlation from Module 1, now anchored to a topology. This makes the map a launchpad, not just a picture.

### Exam focus

Be able to (1) state that the map is generated from trace data automatically and reflects real, current dependencies; (2) explain upstream vs downstream and the left-to-right convention; (3) interpret border colours via Service Health; and (4) describe the culprit-vs-victim reasoning for root-cause analysis.

**Common dependency-analysis questions the map answers:** Which services depend on the one I'm about to change (blast radius)? Where is the latency bottleneck in a request path? Is a dependency shared across teams (a single point of failure)? Are there unexpected cross-environment calls (e.g. a prod service

reaching a staging dependency)? Which node, when it fails, turns the most upstream services red (criticality)?

### 3.3 Labs, use cases & review questions

#### Use case — Cascading failure triage

At 03:10 pagers fire for `web-store`, `mobile-api`, and `checkout` — three red nodes. Panic suggests three problems. The Service Map shows all three depend, directly or transitively, on `payments`, which is also red, and `payments`' only red downstream is `postgres`. Following the red downstream, the on-call finds `postgres` is the deepest red node whose dependencies are healthy — the origin. The upstream reds are victims of the same root cause (a saturated DB connection pool). One fix clears all four. *Lesson:* the map converts "five alerts" into "one root cause."

#### Lab 3.1 — Identify the culprit

*Goal: apply culprit-vs-victim reasoning.*

Using Figure 3.1: `checkout` shows a yellow (warning) border; `payments` shows red; `auth` and `cart` are green; `postgres` (downstream of `payments`) is shown as a datastore with no health alert. Which node is the most likely root cause, and how would you confirm it?

#### Solution

`payments` is the most likely culprit: it is the deepest red node, its sibling dependencies ( `auth` , `cart` ) are green, and its own downstream datastore ( `postgres` ) shows no alert — so the failure originates *in* `payments` , not below it. `checkout` is yellow because it depends on the failing `payments` (a victim). Confirm by clicking `payments` to open its error-rate/latency graphs and its error traces; read the correlated logs on those traces to get the concrete exception. If `postgres` were also red you would continue downstream before concluding.

### Lab 3.2 — Assess blast radius before a change

*Goal: use upstream dependencies for change safety.*

You plan to deploy a breaking API change to `cart`. Using the map, describe how you determine who is affected and how you would stage the rollout safely.

#### Solution

Select `cart` and read its **upstream** services (to the left) — every service with an edge *into* `cart` is a caller that could break: here `checkout` (and transitively `web-store` and `mobile-api`). That set is the blast radius. Safe rollout: coordinate with the owning teams (via the Service Catalog), version the change behind compatibility (`DD_VERSION` to track the deploy), release to one environment/canary first, and watch the upstream services' error rates and the map's health borders before proceeding. If any upstream turns yellow/red post-deploy, roll back.

### Lab 3.3 — Spot an architectural smell

*Goal: use the map to find structural risks.*

On a large map you notice one datastore node has inbound edges from eight different services across two teams. Why might this be a risk, and what would you investigate?

#### Solution

A datastore shared by many services is a **single point of failure** and a coupling/contention hotspot: its saturation or an outage turns many upstream services red at once (as in the cascading-failure use case), and schema changes ripple across teams. Investigate its error/latency trends and connection saturation, whether the eight services truly need direct access or should go through a service boundary, and whether it warrants isolation (dedicated pools, read replicas, or splitting the store per bounded context). The map made an invisible coupling visible.

### Review questions — Module 3

- Q17. How is the Service Map generated, and how "current" is it?
- Q18. Define upstream and downstream, and state the left-to-right layout convention.
- Q19. What determines a node's border colour on the map?
- Q20. In root-cause analysis, how do you distinguish a "culprit" service from a "victim"?
- Q21. How would you use the map to assess the blast radius of a change to a service?
- Q22. What does the Service Catalog add that the Service Map does not?
- Q23. Why is a datastore with many inbound service edges an architectural risk?

**Q24.** A service disappears from the map. Give a benign reason this can happen.

**Q25.** Name two cross-pillar pivots you can launch directly from a node on the map.

Answers in Appendix A.



# Appendix A — Answer keys

## Module 1

| #  | Answer                                                                                                                                                                                                                                                       |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q1 | Monitoring watches a predefined set of signals and alerts on known conditions ("that" something is wrong); observability lets you infer internal state from outputs and ask new questions about unforeseen problems ("why"), without shipping new code.      |
| Q2 | Metrics — <i>what</i> is happening (trend/magnitude); Logs — <i>why</i> a specific event happened; Traces — <i>where</i> in the request path the problem is.                                                                                                 |
| Q3 | Each unique tag combination is a separate metric time series (costly and slow at high cardinality), whereas traces store per-request context natively, so high-cardinality attributes like <code>user_id</code> are a natural fit.                           |
| Q4 | Traces are the most sampled pillar. Trend accuracy is preserved because the agent computes trace <i>stats</i> (hits, errors, durations) on 100% of spans <i>before</i> sampling affects storage.                                                             |
| Q5 | SLI = a measured signal (e.g. success ratio); SLO = a target for that SLI (e.g. 99.9%); error budget = the allowed shortfall (0.1%) teams may spend on risk. The SLO sets the goal on the SLI; the budget is what remains before action is required.         |
| Q6 | Any two: traces localise the failing service/operation instantly; flame graphs quantify where time is spent; correlated logs explain the failure; the service map turns many alerts into one root cause — all faster than inferring from host metrics alone. |

## Module 2

| #   | Answer                                                                                                                                                                                                                                 |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q7  | Receive → Normalize → Obfuscate → Sample → Aggregate (stats) → Write.                                                                                                                                                                  |
| Q8  | Because trace stats are computed on every span by the concentrator before the sampling decision affects what is stored for search.                                                                                                     |
| Q9  | <code>trace_id</code> , <code>span_id</code> , <code>parent_id</code> , service, resource (plus start time and duration for timing).                                                                                                   |
| Q10 | Head-based: decide keep/drop at the root span and propagate the decision. Tail-based: buffer the whole trace, then decide using error/latency-aware policies.                                                                          |
| Q11 | "Keep all traces slower than X" — total trace latency is unknown at the root, so only tail-based (which buffers the full trace) can decide on it.                                                                                      |
| Q12 | <code>env</code> (scope/isolate environments), <code>service</code> (identify the service across pillars), <code>version</code> (deployment tracking / regression comparison).                                                         |
| Q13 | All spans of a given trace must reach the <i>same</i> collector instance, so collectors must load-balance by <code>trace_id</code> .                                                                                                   |
| Q14 | Receivers ingest (e.g. <code>otlp</code> ), processors transform/sample (e.g. <code>batch</code> , <code>tail_sampling</code> ), exporters send onward; the <code>datadog</code> exporter is the final stage that forwards to Datadog. |
| Q15 | The DDOT Collector adds Datadog ecosystem features — Fleet Automation, container monitoring, 1,000+ integrations — on top of OTel flexibility.                                                                                         |
| Q16 | Mismatched trace-context propagation (the boundary service doesn't read/inject the same headers), so the downstream service starts a new trace.                                                                                        |

## Module 3

---

- Q17 Automatically, from aggregated trace data, in real time; services appear within moments and drop off after ~30 days of no traffic.
- Q18 Upstream = services that call the selected service (drawn left); downstream = services/datastores it depends on (drawn right).
- Q19 Service Health, which consolidates Watchdog anomalies, paging monitors, and incidents into one state (green/yellow/red).
- Q20 A culprit is red with healthy downstream dependencies (failure originates there); a victim is red only because a service it depends on is red. Find the deepest red node with green dependencies.
- Q21 Select the service and read its upstream callers — that set is the blast radius; stage the rollout (canary/one env), track with `version`, and watch upstream health before proceeding.
- Q22 Ownership, on-call, SLOs, deployment version, and links to dashboards/monitors/docs — an inventory view versus the map's spatial view.
- Q23 It is a single point of failure and contention/coupling hotspot: its saturation reddens many upstream services and schema changes ripple across teams.
- Q24 It has had no traffic for ~30 days (e.g. decommissioned or idle), so it aged off the map — not necessarily an outage.
- Q25 Any two: to that service's traces/flame graphs, latency & error metrics, correlated logs, underlying infrastructure, or dashboards.
-

# Appendix B — Glossary & quick-reference cheat sheet

## Glossary

| Term                     | Definition                                                                                                                                                                                                   |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| APM                      | Application Performance Monitoring — request-level visibility into an application via distributed traces, service metrics, and profiling.                                                                    |
| Span                     | One timed unit of work in a trace, with <code>span_id</code> , <code>parent_id</code> , <code>trace_id</code> , <code>service</code> , <code>resource</code> , <code>timing</code> , and <code>tags</code> . |
| Trace                    | All spans sharing one <code>trace_id</code> — the end-to-end path of a single request.                                                                                                                       |
| Root span                | The first span in a trace (no parent); origin of head-based sampling decisions.                                                                                                                              |
| Resource                 | The specific operation a span represents (e.g. <code>GET /checkout</code> , a SQL statement).                                                                                                                |
| Context propagation      | Passing trace identifiers + sampling priority across service boundaries (via W3C <code>traceparent</code> , Datadog, or B3 headers).                                                                         |
| Sampling priority        | Propagated keep/drop flag ensuring a trace is retained or dropped as a whole.                                                                                                                                |
| Head-based sampling      | Keep/drop decided at trace start and propagated.                                                                                                                                                             |
| Tail-based sampling      | Keep/drop decided after buffering the full trace; error/latency-aware.                                                                                                                                       |
| Ingestion vs indexing    | Ingestion = data sent to Datadog; indexing (retention filters) = which ingested spans stay searchable (15 days).                                                                                             |
| Trace metrics/stats      | Hits, errors, and duration distributions computed on 100% of spans, independent of sampling.                                                                                                                 |
| OpenTelemetry (OTel)     | Vendor-neutral standard: API, SDKs, OTLP protocol, and Collector.                                                                                                                                            |
| OTLP                     | OpenTelemetry Protocol — the wire format for exporting telemetry (gRPC/HTTP).                                                                                                                                |
| OTel Collector           | Standalone pipeline of receivers → processors → exporters.                                                                                                                                                   |
| DDOT Collector           | Datadog Agent with an embedded OTel Collector (recommended OTel path).                                                                                                                                       |
| Service Map              | Auto-generated graph of services (nodes) and their calls (edges) from trace data.                                                                                                                            |
| Service Catalog          | Inventory of services with ownership, on-call, SLOs, versions, and links.                                                                                                                                    |
| Unified Service Tagging  | The reserved tags <code>env</code> , <code>service</code> , <code>version</code> that correlate telemetry.                                                                                                   |
| SLI / SLO / error budget | Measured signal / target for it / allowed shortfall before action.                                                                                                                                           |
| MTTR                     | Mean time to resolve/recover — reliability KPI that observability lowers.                                                                                                                                    |

## Sampling cheat sheet

| ingestion_reason | Mechanism                     | Key control (default)                       |
|------------------|-------------------------------|---------------------------------------------|
| auto             | Agent target-TPS auto sampler | DD_APM_TARGET_TPS (10)                      |
| rule             | Library sampling rules        | DD_TRACE_SAMPLING_RULES ; rate limit ~100/s |
| error            | Error sampler                 | DD_APM_ERROR_TPS (10)                       |
| rare             | Rare sampler                  | DD_APM_ENABLE_RARE_SAMPLER (~5/s)           |
| manual           | Manual keep/drop              | Span tag / manual.keep                      |
| single_span      | Single span sampling          | DD_SPAN_SAMPLING_RULES                      |

## Key environment variables

| Variable                                      | Purpose                                              |
|-----------------------------------------------|------------------------------------------------------|
| DD_ENV / DD_SERVICE / DD_VERSION              | Unified Service Tagging.                             |
| DD_AGENT_HOST / DD_TRACE_AGENT_PORT           | Where the tracer sends spans (default port 8126).    |
| DD_TRACE_SAMPLING_RULES / DD_TRACE_RATE_LIMIT | Library-side sampling & rate limiting.               |
| DD_APM_TARGET_TPS / DD_APM_ERROR_TPS          | Agent auto and error sampler targets.                |
| DD_TRACE_PROPAGATION_STYLE                    | Which context-propagation headers to inject/extract. |

### One-page memory hooks

**Pillars:** metrics = what, logs = why, traces = where. **Pipeline:** Receive-Normalize-Obfuscate-Sample-Aggregate-Write. **Sampling:** head = decide-then-propagate, tail = buffer-then-decide; latency-based needs tail. **Stats ≠ sampling:** metrics computed on 100% of spans. **Tagging:** env/service/version. **Map:** nodes=services, edges=calls, colour=health, upstream-left/downstream-right, deepest-red-with-green-deps = culprit.

## Appendix C — Sources & further reading

This guide's factual claims about the Datadog Agent pipeline, ingestion mechanisms, OpenTelemetry integration, and the Service Map are grounded in the official Datadog documentation and Agent source references below. Confirm current defaults against the live docs before relying on specific numbers.

- Datadog Docs — APM / Tracing overview: [docs.datadoghq.com/tracing/](https://docs.datadoghq.com/tracing/)
- Datadog Docs — Trace ingestion mechanisms: [docs.datadoghq.com/tracing/trace\\_pipeline/ingestion\\_mechanisms/](https://docs.datadoghq.com/tracing/trace_pipeline/ingestion_mechanisms/)
- Datadog Docs — Ingestion controls & volume control: [docs.datadoghq.com/tracing/trace\\_pipeline/ingestion\\_controls/](https://docs.datadoghq.com/tracing/trace_pipeline/ingestion_controls/)
- Datadog Docs — OpenTelemetry with Datadog: [docs.datadoghq.com/opentelemetry/](https://docs.datadoghq.com/opentelemetry/)
- Datadog Docs — Ingestion sampling with OpenTelemetry: [docs.datadoghq.com/opentelemetry/ingestion\\_sampling/](https://docs.datadoghq.com/opentelemetry/ingestion_sampling/)
- Datadog Docs — Service Map: [docs.datadoghq.com/tracing/services/services\\_map/](https://docs.datadoghq.com/tracing/services/services_map/)
- Datadog Agent — Trace Agent architecture (component reference): [github.com/DataDog/datadog-agent](https://github.com/DataDog/datadog-agent)
- OpenTelemetry — Collector & tail-sampling processor: [opentelemetry.io/docs/collector/](https://opentelemetry.io/docs/collector/)

---

Study guide compiled August 2026. Diagrams are original, produced for this guide. UI screenshots are intentionally left as labelled placeholders for you to capture live from your own Datadog account so your copy stays current.